

VIRASAT

Team guide

Six parts, one per person.
What it does, how it works, and what you will be asked.

Smart India Hackathon 2026 / internal round
Heritage and Culture
sih26-team-sigmoid.vercel.app

Before anything else

How to use this

The host can ask anyone about anything. This document splits the project into six parts so each person owns one properly, but everybody should read the two pages that follow this one, because those answer the questions that get asked most and they are about the project as a whole rather than any one part.

The three sentences everyone must be able to say

- **What it is.** VIRASAT reads century old survey records of Indian monuments, works out where the buildings they describe are on today's map, has a person confirm each one, and then lets those places narrate their own history to a visitor standing in front of them.
- **The one rule.** Show the evidence, or do not show it at all. Every claim on screen has a route back to its source in one step. If a feature could not show where its claim came from, we did not build it.
- **What is different.** We do not generate heritage. A model helps read old documents, but nothing it produces is shown as fact until a human being approves it, and every sentence a visitor hears was written from a sourced Fact Sheet and reviewed before it shipped.

The sentence that wins the room

We take India's forgotten heritage out of the archive, put it back on the map, and let people stand where it happened.

The six parts

Part	What it covers	Main files
1. The Archive	Getting real scanned pages of a 1919 survey into the project	scripts/ingest-volume.ts, content/pages/
2. Reading the page	Turning a page of old English into structured Mentions	lib/ai/model.ts, lib/discovery/extract.ts
3. Putting it on the map	Turning a written direction into a pin and a circle of doubt	lib/discovery/resolve.ts, baseline.ts, confidence.ts
4. The human check	Why nothing automated is allowed to claim a monument exists	lib/store/, app/authority/
5. The Walk	Why a place starts speaking when you arrive and face it	lib/location/engine.ts, lib/route/planner.ts
6. Content and proof	The words, the voices, the images and where they all came from	content/, scripts/render-audio.ts, app/attributions/

Everybody reads this

The questions that get asked most

Q. Is this just ChatGPT with a map?

No, and the difference is structural. A model is used in exactly one place: reading an old page into structured fields. Everything it returns is checked against the original scan before it is used, and a passage the model wrote instead of copying is thrown away. Two of our 123 extracted Mentions were rejected that way. After that point no model is involved. The location maths, the confidence score, the review workflow and the narration are all ordinary code and human writing.

Q. So the AI discovered a monument?

We never say that and the interface never says it. The archive recorded it, we projected it onto today's map, and a Reviewer confirms it. The furthest the automated pipeline may go is proposing a Candidate. Saying a monument exists is a human decision, on the Authority page.

Q. How do you know the pin is in the right place?

We do not know exactly, and that is the point. Every pin is drawn with an Uncertainty Radius, a circle whose size is calculated from four things: how precisely the landmark is known, how vague the compass direction is, how vague the distance unit is, and a floor. A vague clue produces a wide circle. A wide circle is an honest answer; a bare pin would be a lie.

Q. What happens if the internet or the AI fails during the demo?

Everything keeps working and the screen says why. Reading a page falls back to a copy committed with the app and prints the reason, for example that both providers returned a rate limit. The Candidate queue falls back to a snapshot and shows a stale label. The whole Walk, all 36 audio clips and the trigger logic are on the device already. We tested all of this on purpose by breaking each service.

Q. Why only Delhi and only one fort?

Because depth is the proof. The pipeline is not Delhi shaped, the content is. Adding a city means ingesting another volume and writing content, not changing the system. In one week we chose one site done properly over five done badly.

Q. What is a Representation Gap?

A structure the 1919 survey clearly recorded, whose position we resolved, and where nothing in today's OpenStreetMap data sits inside the circle. In plain words: the archive knew about it and the modern map does not. We found five in forty pages. That is the headline finding.

Q. Is the narration generated live by AI?

No. Every narration was written from a Fact Sheet of sourced lines, reviewed, and rendered to an audio file that ships with the app. At a Threshold Crossing the app plays a file. Nothing is generated while a visitor is standing there, because an unreviewed sentence about a real monument spoken aloud to a historian is exactly the risk this project exists to avoid.

Part 1

The Archive

Owned by: the ingestion and source owner

In one sentence: get real pages of a real 1919 survey into the project, as both a picture and as text, so that everything downstream is arguing about a document a judge can look at.

What it actually is

Maulvi Zafar Hasan's *List of Muhammadan and Hindu Monuments, Delhi Province* was published by the Archaeological Survey of India between 1916 and 1922. It is a monument by monument inventory. Volume 2 covers the areas outside the walled city. It is public domain and it is on archive.org.

How it works

- **scripts/ingest-volume.ts** downloads a page image and the matching page text for each chosen page, and writes them into **public/pages/** and **content/pages/**.
- The text comes from the archive's own OCR file rather than us running OCR. That is deliberate: it is the archive's transcription, so it is theirs to be wrong, and it is reproducible.
- We ingested 40 pages, chosen for how many location clues they contain.
- The scans are served from our own site at runtime, never hot linked from archive.org, so the demo does not depend on somebody else's server being up.

The decision worth knowing about

Why Volume 2 and not Volume 1

Volume 1 covers Shahjahanabad, the walled city, where the surveyor locates a building by naming the neighbourhood it stands in. That cannot be turned into a coordinate.

Volumes 2 and 3 cover the outlying areas, where there is no neighbourhood to name and the surveyor has to measure: *Some 300 yards from the Police Station Paharganj, towards south*. That form is the only one our resolver can work with.

We counted: Volume 1 has 1 measured clue in 231 pages. Volume 2 has 213 across 127 pages. Volume 1 is not a weaker source, it is the wrong one for this job. It is still our source for the Red Fort Fact Sheets, which is what it is good at.

Questions you will get

Q. Is the document real?

Yes, and it is on screen. The Discover page shows the actual scanned page beside its text, and links to the archive.org record. You can zoom into the page and read the same sentence the system just extracted.

Q. Did you do the OCR?

No. We use the transcription archive.org already publishes with the scan. We keep it character for character, including its mistakes, because our passage check compares the model's output against this text and tidying it would break that check.

Q. Why only 40 pages?

Time, and rate limits on the free model tier. The number is not a technical ceiling. The script takes a page range.

Part 2

Reading the page

Owned by: the extraction owner

In one sentence: turn a page of 1919 English into a list of structures, each with a name, a type, a period and the sentence that says where it stood, and refuse anything the page does not actually say.

How it works

```

page text
  |
  v
lib/ai/model.ts      Groq first, then Gemini, both with an 8 second timeout
  |                  asks for JSON matching a strict schema
  v
lib/discovery/extract.ts
  |                  checks every returned passage against the real page text
  v
Mention[]            name, type, period, passage, passageOffset, spatialClue

```

- **One place reads a model key.** `lib/ai/model.ts` is the only file in the project that touches `GROQ_API_KEY` or `GEMINI_API_KEY`, and it is server only. Nothing else can call a model.
- **Two providers, then a cache.** Groq runs first, Gemini second, and if both fail we serve a copy of that page's result committed with the app, labelled **cached** with the reason printed underneath.
- **Plain fetch, not an SDK.** Both SDKs default to a 60 second timeout and retry on their own. The whole design turns on giving up at 8 seconds, so we call the HTTP API directly.
- **Strict JSON schema.** The model is not asked to write prose. It is asked to fill fields, and the response is validated with zod before anything touches it.

The guard that matters most

The model must copy the sentence that locates the building, character for character. We then search for that sentence in the real page text. If it is not there, the model composed it, and we place nothing and say so in red on the card.

This caught a real case: on one page the model returned *Some 250 yards from the south gate of the Purana Qila* where the page actually reads *Some 80 yards from the S. E. corner*. 2 of our 123 Mentions are refused this way.

The search allows for the survey breaking words across lines with a hyphen, so *hori-* plus *zontal* still matches *horizontal*, and it maps the position back to where it really sits in the page so the highlight lands correctly.

Questions you will get

Q. What if the model hallucinates?

Then the passage will not be found in the page and nothing gets placed. That is the guard above, and we can show it firing on a real example.

Q. Why two AI providers?

Free tiers are unreliable. We measured both returning rate limit errors about half the time under load. Groq is faster, Gemini is the backup, and the committed cache is the third layer. The demo is unchanged except for one honest label.

Q. Is the extraction deterministic?

Not perfectly, even at temperature zero. The number of structures found is stable in our testing; the exact wording of a name can vary between providers. That is why the demo never promises a specific output from the live path.

Part 3

Putting it on the map

Owned by: the geospatial owner

In one sentence: turn a written direction like *300 yards south of the Police Station Paharganj* into a coordinate and a circle that is honest about how unsure we are.

The three pure modules

Module	What it does	Tested
lib/discovery/resolve.ts	Spatial Clue plus Anchor into a coordinate and an Uncertainty Radius	yes, first
lib/discovery/baseline.ts	Is there anything in today's OpenStreetMap data inside that circle	yes, first
lib/discovery/confidence.ts	A score in five named parts, always shown broken down	yes, first

These three take data and return data. No network, no database, no files. That is why they can be tested in milliseconds and why they were written test first.

How the circle is calculated

Uncertainty Radius = anchor precision	how tightly the landmark is known
+ bearing spread	an eight point compass is coarse
+ distance vagueness	a kos is anywhere from 1.8 to 3.2 km
+ a floor	nothing is ever claimed to be exact

Those four numbers are shown separately in the Evidence panel, not just their total. A judge can see which part of the doubt is doing the work.

Anchors

- 62 landmarks that the volume measures distances from, each with a coordinate, alternative spellings the volume uses, and a precision in metres.
- Every one carries a **source**. Usually an OpenStreetMap id such as osm:w223456559. Two say **approximate** because nothing on the modern map corresponds to them, and the screen says so.
- They were built from the corpus, not from a list of famous places: all 69 location lines in our 40 pages were extracted first and the table written to match.
- If a clue names a landmark we do not have, the Mention is shown as unresolvable with its passage. It is never given a guessed coordinate.

The verdict has three values, not two

matched_existing: something is already on today's map inside the circle. That is the most persuasive result, because it proves the pipeline is not inventing pins.

representation_gap: the circle is tight enough to be meaningful and nothing modern is inside it. The archive knew, the modern map does not.

inconclusive: past about 500 metres the circle covers so much of Delhi that finding something proves nothing and finding nothing proves nothing. Saying so is more honest than forcing a two way answer.

Questions you will get

Q. A kos is not a precise unit.

Correct, it varied a lot, and that is exactly why the radius is wide for a clue measured in kos. The vagueness of the unit is one of the four inputs to the circle.

Q. What if the pin lands in a river?

It can, and the circle is why that is survivable. A wide circle over a river is a visible statement that the clue was not good enough, which is a true statement.

Q. Where does the modern map come from?

One Overpass query against OpenStreetMap, pulled once, 585 features, committed to the repo with the query text and the pull date beside it so anyone can rerun it.

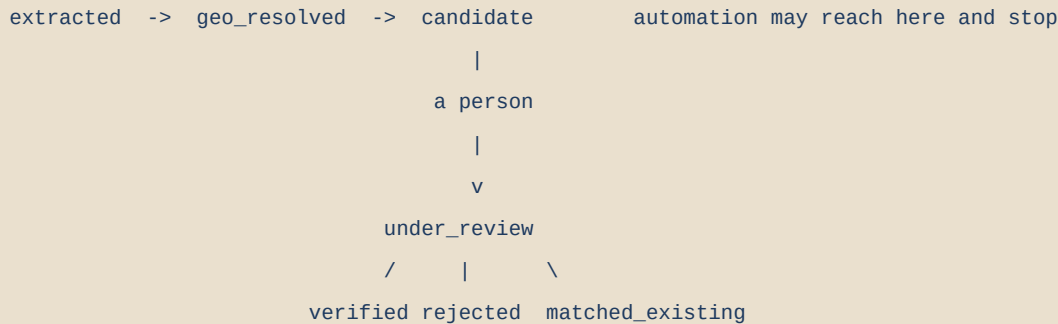
Part 4

The human check

Owned by: the review and data owner

In one sentence: nothing automated is allowed to say a monument exists, and this is the part of the system that enforces it.

The rule



The transitions are one exported table in **lib/store/transitions.ts**, tested, and checked on the server before any write. The states after candidate are unreachable except through a human pressing a button on the Authority page.

Why the confidence score never promotes anything

It would be easy to auto verify anything scoring above 0.9 and it would make the demo self running. We rejected it. The entire credibility of the project rests on not overclaiming, and the state where the pipeline finds something already documented is the most persuasive evidence that it is not inventing. Confidence is a number displayed beside a Candidate. It is never a state and never a trigger.

How the data is stored

- Postgres, on Supabase, holds only what changes: Candidates, their review state, and Walk logs.
- Everything else, sites, points, fact sheets, narrations and audio, is committed to the repository and read at build time. Every screen works with the database switched off.
- **candidate_events** is append only. Every status change writes a row with where it came from, where it went, and when.
- All Supabase access lives in **lib/store/**. Nothing else in the codebase imports the Supabase client.

A trap we fell into, worth telling

A Postgres update that matches zero rows is not an error. It reports success. We had a case where a review move changed nothing and still wrote an audit event saying it had.

The update now selects what it changed and refuses when that is nothing. If the event row cannot be written, the status change is rolled back, because a status that moved with no record of how it got there is exactly what that table exists to prevent.

Questions you will get

Q. There is no login on the Authority page.

Correct, and it is documented rather than hidden. For the internal round anyone with the URL can review. The database permissions allow only status updates and event inserts, never deletes, and the events table is append only so tampering is visible. Adding a login is one Supabase email link and was deliberately not this week's hour. We do not claim it is secured.

Q. What if Supabase is down during the demo?

The queue renders from a snapshot committed with the app with a stale label. A review move returns an error and says plainly that nothing was recorded. We deliberately do not fake the badge, because showing verified over a decision that was never saved is the exact thing this page exists to prevent.

Part 5

The Walk

Owned by: the location and routing owner

In one sentence: a place starts speaking on its own when the visitor is close enough, facing it, and has stood still, and the screen shows why at every moment.

Why not just proximity

Proximity alone is what most location apps do and it fails exactly where it matters. At the Red Fort the halls sit tens of metres apart, so a visitor standing between two of them is inside both circles. And a visitor walking past a building on their way somewhere else gets interrupted by it. Facing decides which building the visitor means. Standing still decides whether they have arrived or are passing through.

The five conditions

Condition	What it means	Default
Inside the Approach Ring	Within a buffer around the building's real footprint	25 m buffer
Facing	Heading is within tolerance of the bearing to the building	60 degrees
Dwell	Both held continuously without drifting	3 seconds
Re-arm	Must leave and stay out before it can fire again	10 m for 3 s
Once per Walk	A place does not repeat itself unless you leave and return	-

The engine is a pure function. It takes the current position and the prepared geometry and returns the new state, any crossings, and a status line per building. It does no input and output of any kind, which is why it is fully tested.

The condition that must never fail silently

A trigger that fires wrongly is survivable. A trigger that never fires is fatal, and this rule fails silently by nature.

So the panel on screen names the failing condition at all times: too far away, facing away by 84 degrees, still moving, 1.4 seconds more. If it does not fire on stage we can read the reason out loud and fix it in front of the room.

If the heading is unavailable, the facing condition switches off rather than blocking, and the screen says so in those words.

Simulated and real positions

- A position is a position. The simulator and the phone produce the same shape of data and feed the same engine. Only a source field differs.
- Real GPS wanders several metres while a phone sits still, and standing still is precisely when the dwell timer runs, so a raw feed would mean a crossing that can never fire. The position is smoothed in the location source, not the engine, because the noise belongs to the reading.
- **Bring it here** slides the whole site under the visitor's real position so live tracking can be proved anywhere. Only the coordinates are translated. The satellites, the compass, all five conditions and the audio stay real, and the screen says which half is which.

The Route

A separate tested module. It scores each Heritage Point by importance and how well it matches the chosen interests, fits as many as possible into the time budget including walking time, and then orders them along the shortest path. It reports what it left out. The route is budgeted on the longest telling, so choosing a different narrator changes how a place is told and never which places you visit.

Part 6

Content and proof

Owned by: the content and presentation owner

In one sentence: the words a visitor hears, the voices that read them, the photographs beside them, and the page that says where every one of those came from.

How a narration is made

```

Fact Sheet          sourced lines, each with its citation
|
v
Narration text      written by a person from those lines, three tellings per place
|
v
scripts/render-audio.ts  Edge TTS, Indian English voices, offline, once
|
v
mp3 + timing cues     shipped with the app, transcript follows the voice

```

- 11 Heritage Points, 3 tellings each, plus 3 extra tellings for going inside. 36 clips.
- Nothing is generated while a visitor is present. The app plays a file.
- The transcript is synchronised sentence by sentence. That is real accessibility, and it is also why the demo survives a room with no working speakers.

The check that keeps it honest

`scripts/check-content.ts`

One command validates every piece of content against a schema: that a site marked deep actually has places to walk between, that a building's outline is a closed shape, that coordinates are the right way round, that every narration has a rendered audio file on disk, and that the text still matches what was spoken.

That last check earns its place. When we rewrote eleven narrations, it flagged all eleven as drifted from their audio within a second, so the transcript could never end up describing something the voice was not saying.

Living Tradition

Every one of the 11 Heritage Points carries a practice as well as a building: the drums that kept the hours at the Naubat Khana, the stone inlay at the Diwan-i-Khas that workshops in Agra still do by hand, the covered bazaar at the Chatta Chowk that is still a bazaar. Six are still practised, three survive as crafts but not in that use, and two have stopped. This is the part of heritage a photograph cannot hold, and it is what makes this a culture project rather than a monument list.

Then and Now, and licensing

- Four Heritage Points have a draggable divider between an archival image and a modern photograph of the same building.
- The strongest pair is the Naubat Khana: John Murray photographed it in 1858 head on, and Jakub Halun photographed it from the same spot in 2019. Same building, 161 years apart, and the arcaded court that surrounded it has gone.
- Everything is from Wikimedia Commons. Archival photography is public domain by age, modern photography is Creative Commons and credited at the image and again on the attributions page.

- **No image of a real monument in this project was generated.** The attributions page says so in those words, and it lists every author, date and licence with a link to the file.

Questions you will get

Q. Are you allowed to use these photographs?

Yes, and the attributions page proves it. Archival photography from 1858 is public domain by age. Modern photographs are Creative Commons with attribution, and the photographer, date and licence appear both under the image and on that page.

Q. Did AI write the narration?

A model helped draft from the Fact Sheets, but every sentence was checked against the sourced lines by a person before it was rendered, and the whole clip ships as a file. There is one line we know is our inference rather than the survey's, and it is written down as such in our status document.

[Last page](#)

If something breaks on the day

What breaks	What you say and do
The place will not start speaking	Read the failing condition off the panel out loud. It is telling you why. Every place is also tappable in the list, so manual playback always works.
Analyse comes back cached	Good. Say it out loud: both model providers are rate limited, it says exactly that on screen, and it still answers from the copy shipped with the app. A system that says why it fell back is a better answer than one that never falls back.
The queue shows a stale label	The database is asleep. The queue is rendering from the snapshot committed with the app. Everything is visible, only saving is lost.
No sound in the room	The transcript carries the whole narration and follows sentence by sentence. Say that this is the second reason it exists.
A page places nothing	That is honest output, not a failure. The survey measured that one from a landmark our table does not hold, and the screen says so rather than guessing. Pick a page with a number beside it on the shelf.

What a judge should remember

Not that it was an AI tourism app. That it was the one that took old Indian archival records, found forgotten heritage on the map, and then let you walk through the history.

VIRASAT / Stand where it happened.